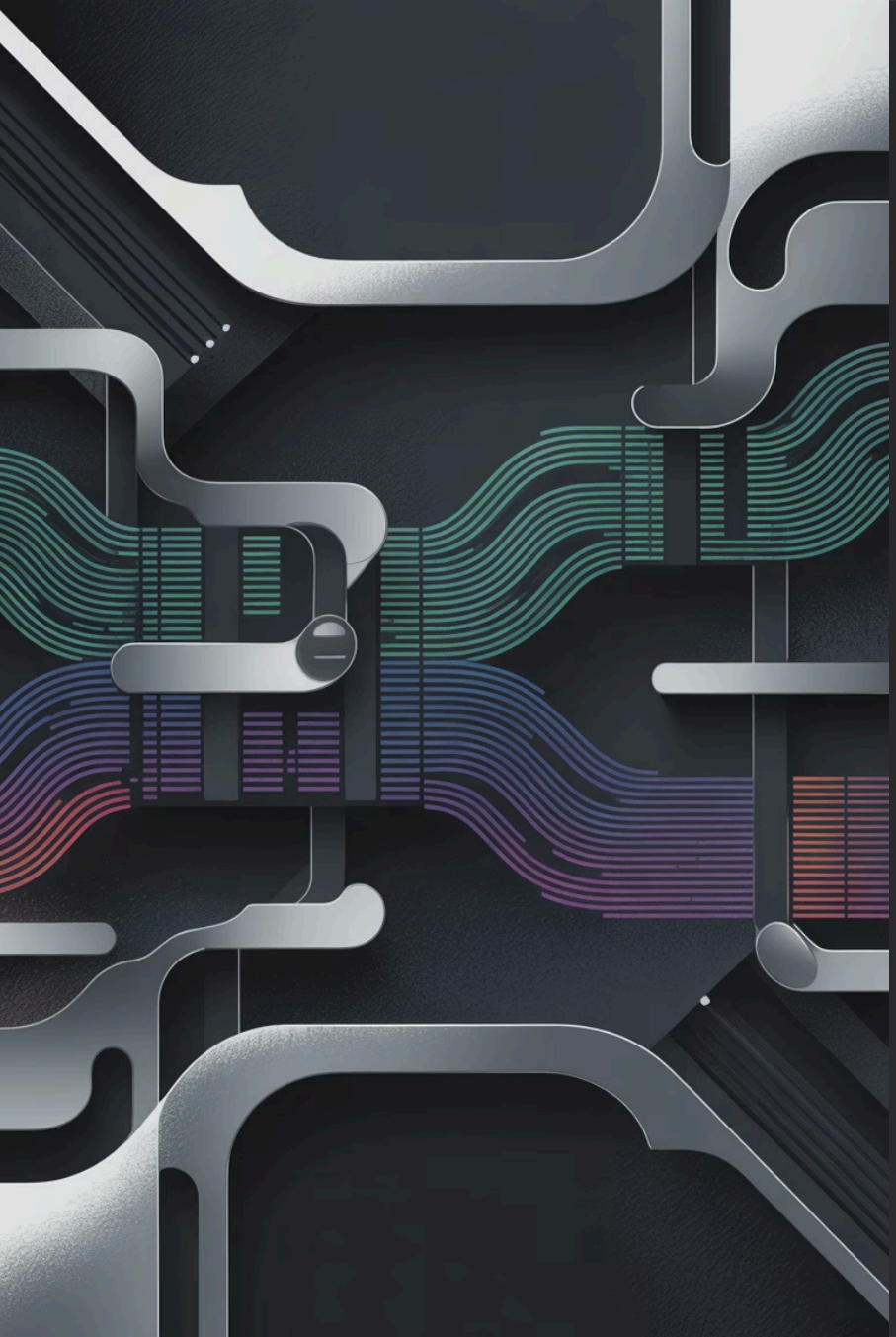


Module 05 - Prompt Engineering & Task-to-Prompt Mapping

Building the Foundation for AI Engineering Excellence

Today's Learning Journey

01

Zero-Shot vs Few-Shot

Understanding the fundamental differences and when to apply each approach

02

Example Curation

Selecting and structuring demonstrations that maximize model performance

03

Ordering Strategies

How example sequence impacts output quality and consistency

04

Counter-Examples

Using negative examples to sharpen model boundaries

05

Hands-On Lab

Building production few-shot classifiers for sentiment and entity extraction

The Power of Learning by Example

How Humans Learn

When teaching someone to identify spam emails, you don't just describe the rules—you show examples. "Here's a phishing attempt... here's a legitimate newsletter... notice the difference?"

Few-shot prompting applies this same intuitive teaching method to LLMs, providing demonstrations that guide the model toward desired behavior patterns.

How LLMs Learn

Large language models excel at pattern recognition. When you provide carefully selected examples in your prompt, the model infers the underlying task structure and generalizes to new inputs.

This in-context learning happens without updating model weights—making it fast, flexible, and production-ready.

Zero-Shot vs Few-Shot: A Critical Distinction

Understanding when to provide examples versus relying on instructions alone is fundamental to effective prompt engineering. The choice impacts accuracy, consistency, and computational cost.

Zero-Shot Prompting

The model receives only task instructions without examples. Best for well-known tasks, simple classifications, or when example collection is impractical.

Example: "Classify the sentiment of this movie review as positive or negative."

Few-Shot Prompting

The prompt includes 2-10 demonstration examples showing input-output pairs. Essential for nuanced tasks, domain-specific patterns, or when consistency is critical.

Example: Providing 5 labeled movie reviews before asking the model to classify a new one.

When Zero-Shot Falls Short

Ambiguous Task Definitions

Terms like "toxic," "relevant," or "high-quality" mean different things in different contexts. Zero-shot struggles without concrete examples defining your specific interpretation.

Complex Output Formats

When you need structured JSON, specific field names, or particular formatting conventions, examples clarify expectations better than descriptions.

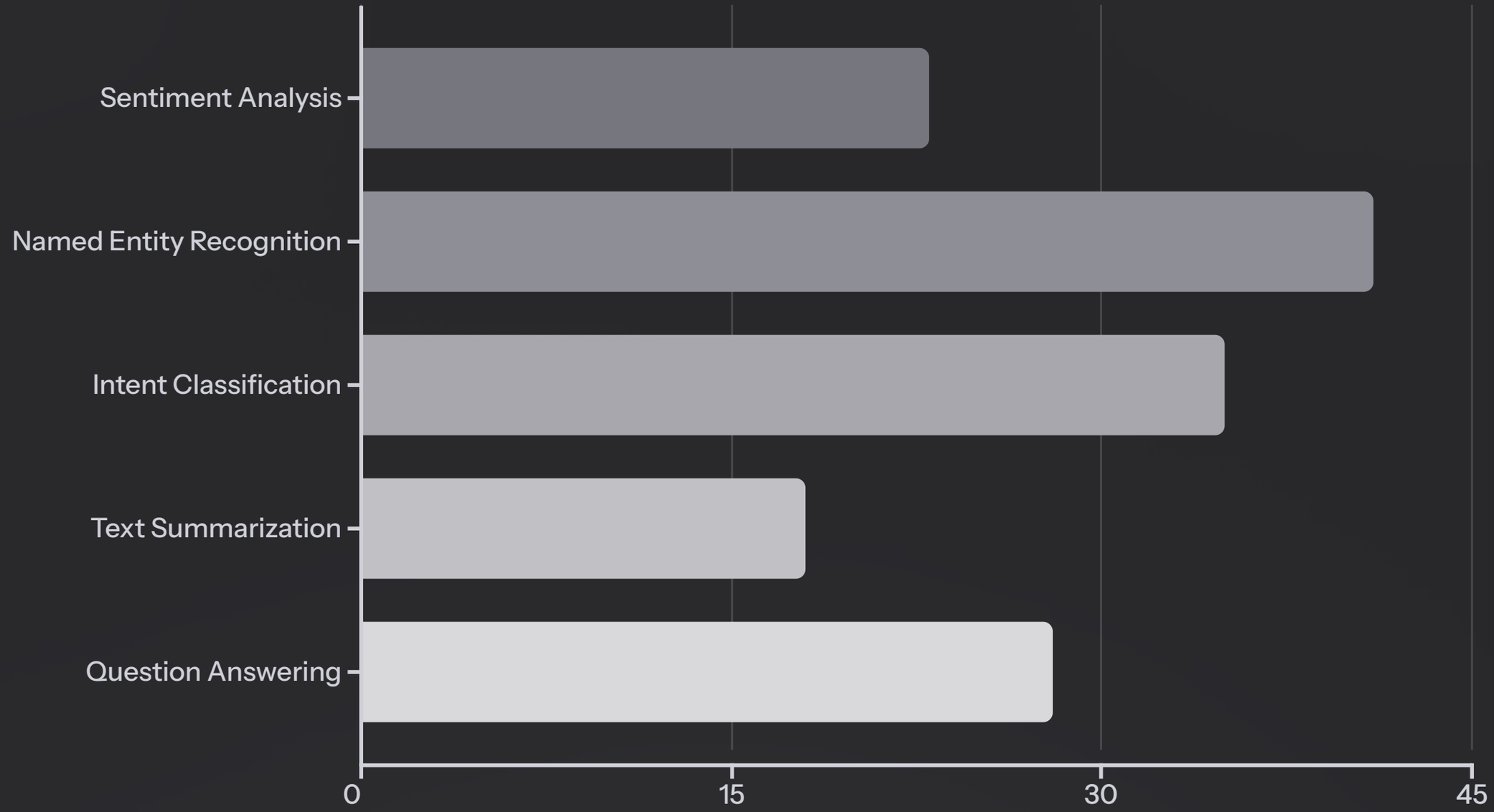
Domain-Specific Patterns

Medical entity extraction, legal document classification, or technical support routing require domain knowledge best conveyed through examples.

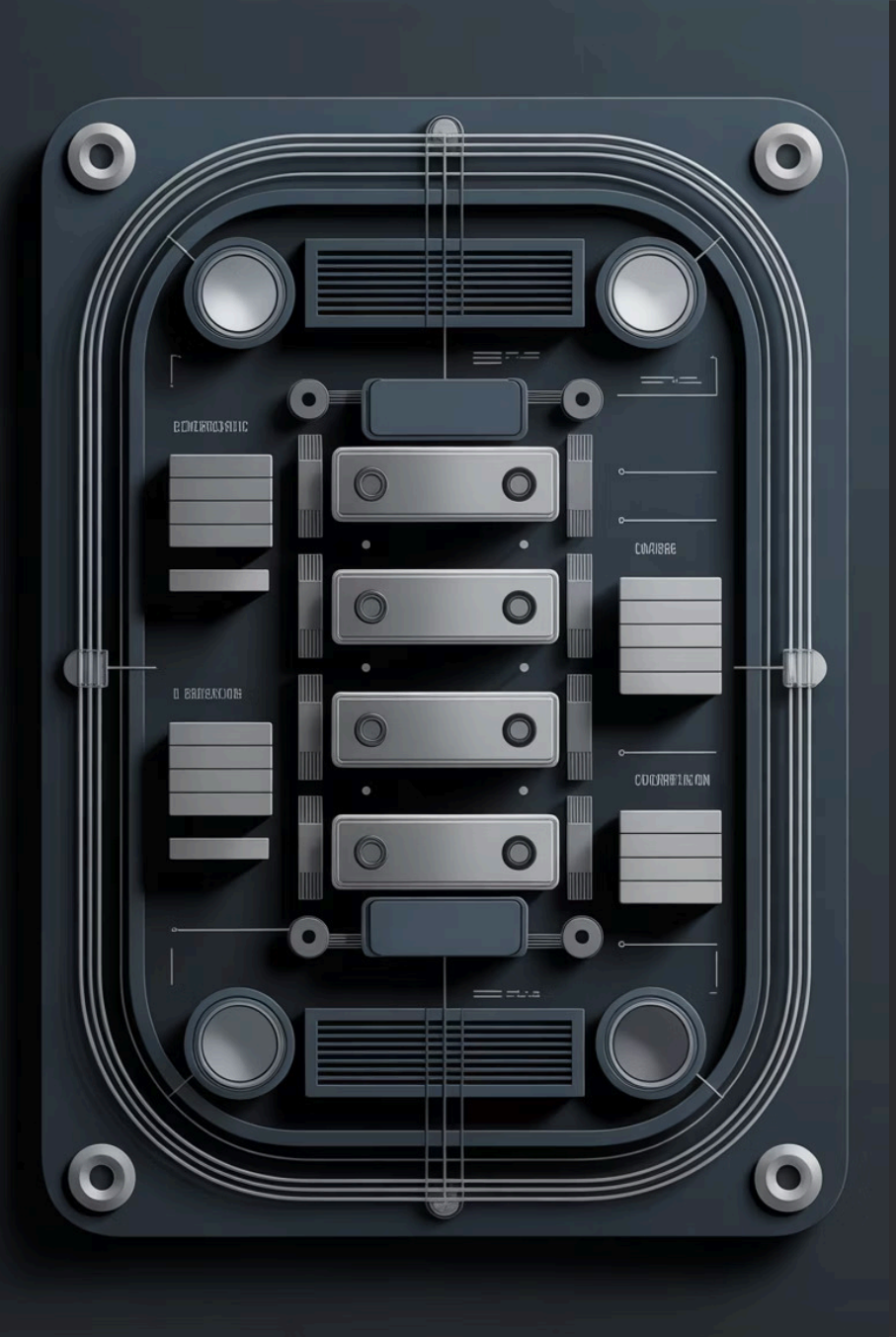
Edge Cases and Boundaries

Where does constructive criticism become negativity? Examples help establish these fuzzy boundaries more precisely than instructions.

The Few-Shot Advantage: Quantitative Impact



Research shows few-shot prompting delivers substantial accuracy gains over zero-shot approaches, particularly for nuanced classification and extraction tasks. The improvement varies by task complexity and domain specificity, with structured extraction tasks seeing the most dramatic benefits.



Anatomy of a Few-Shot Prompt

A production-ready few-shot prompt follows a consistent structure that maximizes model comprehension and output quality. Each component serves a specific purpose in guiding the model's behavior.

Task Description

Clear, concise explanation of the overall objective

Demonstration Examples

2-10 input-output pairs showing the desired pattern

Formatting Instructions

Explicit guidance on output structure and constraints

New Input

The actual data point to process, clearly marked

Example Curation: Quality Over Quantity

The examples you choose have more impact on performance than the number of examples. Strategic curation focuses on diversity, representativeness, and clarity—not simply adding more demonstrations.

Coverage Diversity

Include examples spanning the full range of expected inputs, edge cases, and output categories. For sentiment analysis: clear positive, clear negative, neutral, mixed, and sarcastic.

Class Balance

Distribute examples proportionally across categories, or slightly oversample rare but important classes. Imbalanced examples bias model predictions.

Difficulty Distribution

Mix obvious cases with challenging ones. Too easy: model oversimplifies. Too hard: model gets confused. Balance builds robust understanding.

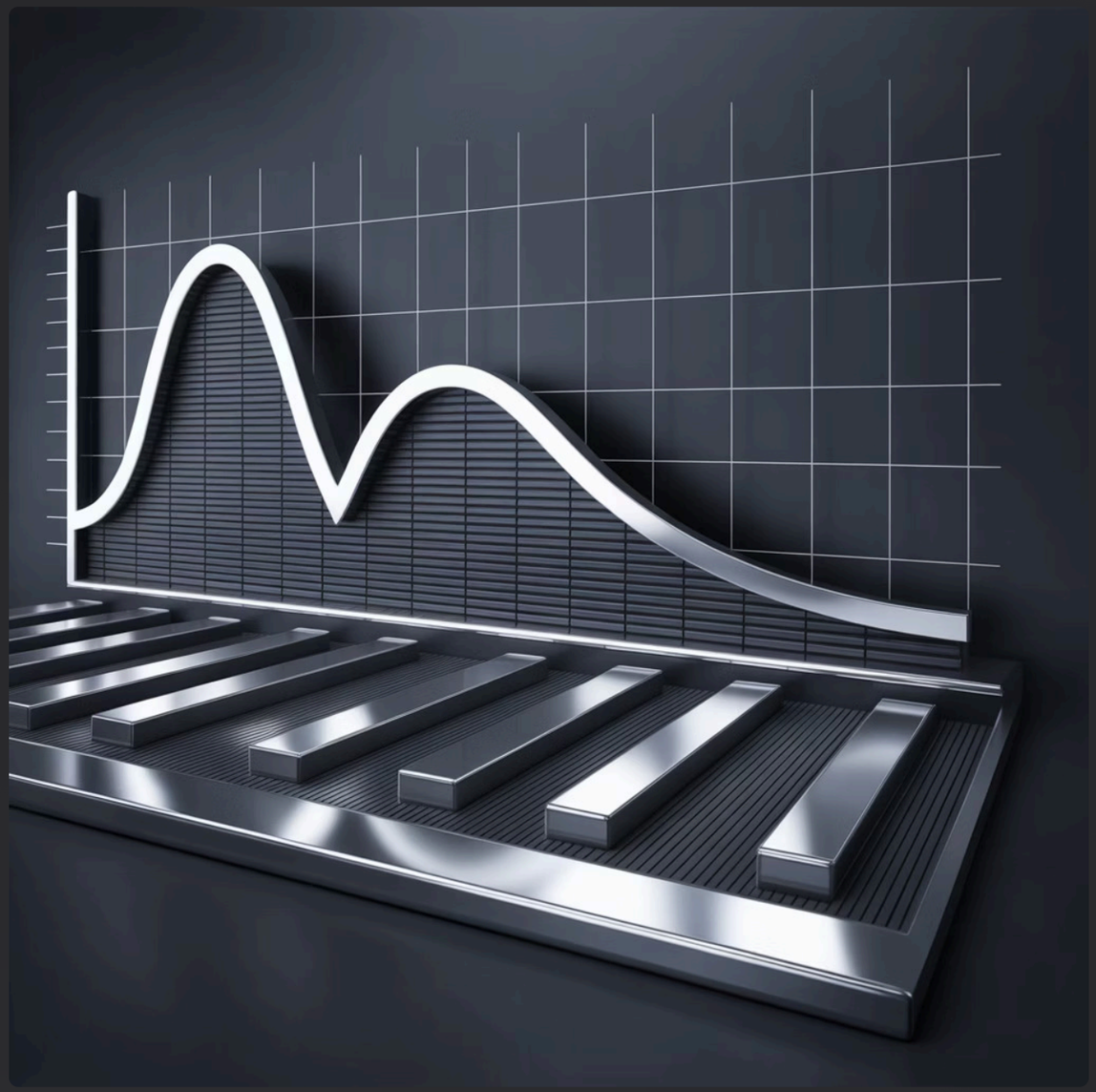
Gold Standard Quality

Every example must be unambiguously correct. A single mislabeled demonstration can corrupt the entire prompt's effectiveness.

The Optimal Number: How Many Examples?

More examples don't always mean better performance. Research and production experience reveal a sweet spot that balances accuracy gains against token cost and latency.

The graph shows diminishing returns beyond 5-7 examples for most tasks, with occasional performance degradation when too many examples create noise or confusion.



2-3 examples: Minimum viable

Sufficient for simple, well-defined tasks with clear boundaries

5-7 examples: Sweet spot

Optimal for most production scenarios, balancing accuracy and cost

8-10 examples: Diminishing returns

Useful only for highly complex or ambiguous tasks

Example Selection Strategy

Don't randomly sample examples from your dataset. Intentional selection strategies significantly outperform random selection, sometimes by 15-20 percentage points in accuracy.

Random Sampling

- Quick baseline approach
- No domain knowledge required
- Risk of unrepresentative examples
- Inconsistent performance

Stratified Sampling

- Ensures class representation
- Better coverage of categories
- Reduces bias systematically
- Requires labeled data

Diversity-Based Selection

- Maximizes input space coverage
- Uses embeddings for similarity
- Captures edge cases effectively
- Computationally intensive

Performance-Guided Selection

- Iteratively tests combinations
- Optimizes for validation metrics
- Finds synergistic examples
- Time-consuming but powerful

Real-World Example: Sentiment Classification

```
# Task Description
Classify the sentiment of movie reviews as positive, negative, or neutral.

# Examples
Review: "This film masterfully blends stunning visuals with profound storytelling. A triumph!"
Sentiment: positive

Review: "Derivative plot, wooden acting, predictable ending. Waste of two hours."
Sentiment: negative

Review: "Standard action fare with decent effects. Nothing groundbreaking, nothing terrible."
Sentiment: neutral

Review: "Started strong but the third act completely fell apart. Disappointing."
Sentiment: negative

Review: "Thought-provoking and beautifully shot, though the pacing drags in places."
Sentiment: positive

# New Input
Review: "The director took risks that mostly paid off, creating something memorable."
Sentiment:
```

Notice how examples demonstrate edge cases: mixed reviews that lean positive/negative, neutral middle ground, and clear extremes. This coverage helps the model handle real-world ambiguity.

Example Ordering: Does Sequence Matter?

The order in which you present examples has a measurable impact on model performance—sometimes changing accuracy by 5-10 points. LLMs exhibit recency bias and primacy effects, making strategic ordering crucial.

1. **Random Order**

   Baseline approach, inconsistent results

2. **Easy to Hard**

   Builds understanding progressively

3. **Similar to Input**

   Places most relevant examples last

4. **Category Grouping**

   Clusters examples by output class

Four Ordering Strategies Compared

Random Ordering

When to use: Quick testing, establishing baselines

Pros: No preprocessing needed, unbiased starting point

Cons: Leaves performance on the table, inconsistent across runs

Difficulty Progression

When to use: Complex reasoning tasks, nuanced classifications

Pros: Scaffolds learning, reduces confusion on ambiguous cases

Cons: Requires difficulty scoring, may suffer from recency bias

Similarity-Based

When to use: Dynamic prompts where input characteristics vary

Pros: Maximizes relevance, adapts to each new input

Cons: Computationally expensive, needs embedding model

Category Clustering

When to use: Multi-class problems with distinct categories

Pros: Clear pattern demonstration, easy to implement

Cons: Can create artificial boundaries, less effective for overlapping classes

Similarity-Based Ordering: Technical Implementation

Dynamically ordering examples based on semantic similarity to the input query often produces the best results, particularly for diverse input distributions. Here's how to implement it efficiently in production.

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

# Pre-compute example embeddings once, then L2-normalize them.
# (all-MiniLM-L6-v2 already normalizes, but doing it explicitly keeps this
#  correct if the model is ever swapped for one that doesn't.)
example_texts = [ex["input"] for ex in example_pool]
example_embeddings = model.encode(example_texts)
example_embeddings /= np.linalg.norm(example_embeddings, axis=1, keepdims=True)


def select_examples(query, k=5):
    query_embedding = model.encode([query])
    query_embedding /= np.linalg.norm(query_embedding, axis=1, keepdims=True)

    # Both sides are unit vectors, so this dot product IS cosine similarity.
    similarities = np.dot(example_embeddings, query_embedding.T).flatten()

    # Top-k most similar: argsort ascending, take the last k, reverse to descending.
    top_k_indices = np.argsort(similarities)[-k:][::-1]
    return [example_pool[i] for i in top_k_indices]
```

This approach adds ~50-100ms latency but can improve accuracy by 8-15% on specialized tasks.

Counter-Examples: Teaching by Contrast

Counter-examples explicitly demonstrate what NOT to do, helping models understand task boundaries more precisely. They're particularly effective for tasks with common failure modes or ambiguous edge cases.

When to Use Counter-Examples

- Classification tasks with overlapping categories
- Situations where the model consistently makes specific errors
- Tasks requiring distinction between subtle differences
- Scenarios with common false positives

Implementation Approaches

- Include 1-2 negative examples among positive ones
- Explicitly label: "This is NOT an example of X"
- Show common mistakes with corrections
- Balance positive and negative examples appropriately

Counter-Example Pattern: Named Entity Recognition

```
# Task: Extract organization names from text

# Positive Examples
Text: "Microsoft announced quarterly earnings yesterday."
Organizations: ["Microsoft"]

Text: "The research was funded by NASA and the National Science Foundation."
Organizations: ["NASA", "National Science Foundation"]

# Counter-Examples (Common Mistakes)
Text: "She works in marketing at the San Francisco office."
Organizations: []
# Note: "San Francisco" is a location, not an organization

Text: "The CEO said the company would expand globally."
Organizations: []
# Note: "company" is too generic; extract specific organization names only

# New Input
Text: "Apple and Google both attended the Mobile World Congress in Barcelona."
Organizations:
```

Counter-examples clarify what NOT to extract, reducing false positives by explicitly addressing common confusion points.

Formatting Consistency: The Hidden Factor

Input Formatting

Use identical formatting for all example inputs and the new query. Inconsistent prefixes ("Input:", "Text:", "Query:") confuse pattern recognition.

Output Formatting

Maintain exact output structure across all examples. If using JSON, ensure consistent field names, spacing, and punctuation throughout.

Delimiter Consistency

Use the same separators between examples (blank lines, "---", "###"). The model learns these as structural markers.

Label Consistency

Standardize terminology: don't mix "positive/negative" with "good/bad" or "favorable/unfavorable" in the same prompt.

Formatting inconsistencies can reduce accuracy by 5-10%, even with perfect examples. Treat formatting as part of the signal.

Dynamic vs Static Few-Shot Prompts

Static Few-Shot

Uses the same curated examples for all inputs. Simpler to implement and maintain, with predictable token costs and latency.

Best for: Consistent input distributions, well-defined tasks, production systems requiring predictable performance.

Limitations: May not adapt well to edge cases or distribution shifts over time.

Dynamic Few-Shot

Selects examples based on each specific input, typically using similarity search or retrieval mechanisms.

Best for: Diverse input characteristics, evolving datasets, maximum accuracy requirements.

Limitations: Higher computational cost, added latency, requires maintaining an example database.

Building a Dynamic Example Selector

```
import chromadb
from chromadb.utils import embedding_functions


class DynamicFewShotSelector:
    def __init__(self, example_pool, n_examples=5):
        # In-memory client (data is lost on process exit).
        # Use chromadb.PersistentClient(path="...") if you need durability.
        self.client = chromadb.Client()

        # get_or_create_collection is idempotent: re-running this cell won't crash.
        # hnsw:space="cosine" makes ranking correct even for embeddings that
        # aren't unit-normalized (don't rely on the model normalizing for you).
        self.collection = self.client.get_or_create_collection(
            name="few_shot_examples",
            embedding_function=embedding_functions.SentenceTransformerEmbeddingFunction(),
            metadata={"hnsw:space": "cosine"},
        )

        # upsert (not add) so re-running with the same ids overwrites instead of erroring.
        self.collection.upsert(
            documents=[ex["input"] for ex in example_pool],
            metadatas=[{"output": ex["output"]} for ex in example_pool],
            ids=[f"ex_{i}" for i in range(len(example_pool))],
        )
        self.n_examples = n_examples

    def get_examples(self, query):
        results = self.collection.query(
            query_texts=[query],
            n_results=self.n_examples,
        )
        # Chroma returns list-of-lists (one inner list per query); we passed one query.
        return [
            {"input": doc, "output": meta["output"]}
            for doc, meta in zip(results["documents"][0], results["metadatas"][0])
        ]
```

Evaluation Metrics for Few-Shot Classifiers

Measuring few-shot performance requires going beyond simple accuracy. Consider per-class metrics, confidence calibration, and consistency across different example sets.

Accuracy & F1 Score

Overall correctness and harmonic mean of precision/recall. Essential baselines but don't tell the full story for imbalanced datasets.

Per-Class Performance

Break down metrics by category to identify which classes benefit most from few-shot examples and which need more work.

Variance Across Example Sets

Test with multiple randomly selected example sets to measure stability. High variance indicates fragile performance.

Latency & Token Cost

Few-shot prompts are longer. Monitor p95 latency and cost per request to ensure production viability.

Common Pitfalls and How to Avoid Them

1

Example Leakage

Using test set examples in few-shot prompts inflates accuracy artificially. Maintain strict separation between example pools and evaluation data.

2

Mislabeled Examples

A single incorrect demonstration can corrupt the entire prompt. Validate all examples with human review or high-confidence automated checks.

3

Class Imbalance

Overrepresenting the majority class in examples biases predictions. Use stratified sampling to ensure proportional representation.

4

Overfitting to Examples

If examples are too similar to each other, the model may memorize patterns rather than learning the task. Maximize diversity.

Advanced Technique: Contrastive Examples

Contrastive examples present near-identical inputs with different outputs, forcing the model to focus on subtle distinctions. This technique is powerful for fine-grained classification tasks.

```
# Contrastive Pair for Toxicity Detection

Text: "Your argument is weak and doesn't hold up to scrutiny."
Toxicity: not_toxic
Explanation: Critical but constructive feedback

Text: "Your argument is stupid and you don't know what you're talking about."
Toxicity: toxic
Explanation: Personal attack, not constructive

# Another Contrastive Pair

Text: "I disagree with your conclusion based on the data presented."
Toxicity: not_toxic
Explanation: Respectful disagreement

Text: "Anyone who believes this conclusion is an idiot."
Toxicity: toxic
Explanation: Insults those with differing views
```

These pairs teach the model to distinguish constructive criticism from personal attacks by highlighting minimal differences that change the classification.

Chain-of-Thought in Few-Shot Prompts

Combining few-shot examples with chain-of-thought reasoning—showing the model's intermediate steps—dramatically improves performance on complex reasoning tasks. Each example includes not just input/output but the reasoning process.

Standard Few-Shot

Q: If a train travels 120 miles
in 2 hours, what's its speed?
A: 60 mph

Q: A car covers 300 miles in
5 hours. What's its speed?
A: 60 mph

Q: A bike goes 45 miles in
1.5 hours. What's its speed?
A:

Chain-of-Thought Few-Shot

Q: If a train travels 120 miles
in 2 hours, what's its speed?
A: Speed = Distance / Time
= 120 miles / 2 hours = 60 mph

Q: A car covers 300 miles in
5 hours. What's its speed?
A: Speed = Distance / Time
= 300 miles / 5 hours = 60 mph

Q: A bike goes 45 miles in
1.5 hours. What's its speed?
A:

Production Considerations

Example Versioning

Track which example sets are deployed in production. Version them like code—changes to examples can impact accuracy as much as model changes.

Caching Strategies

For static few-shot prompts, cache the prompt prefix to reduce token costs. Only the new input needs to be processed per request.

Example Refresh Strategy

As your data distribution evolves, periodically update your example pool. Schedule quarterly reviews to ensure examples remain representative.

Fallback Mechanisms

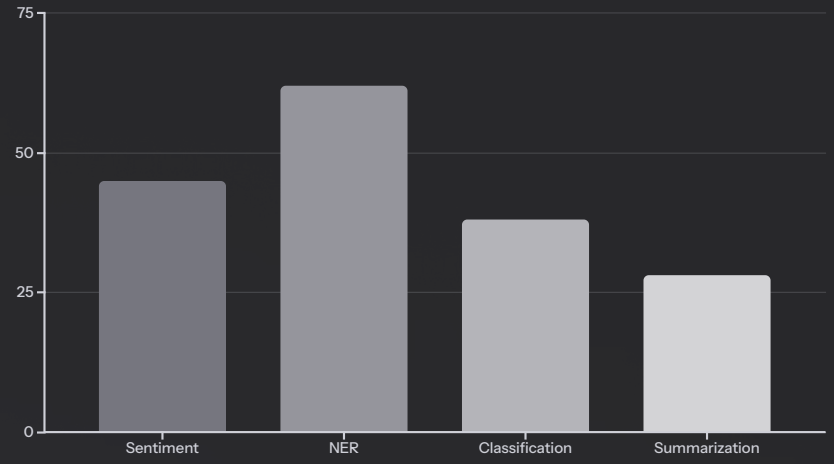
If dynamic example selection fails or times out, have a static fallback set ready. Don't let retrieval failures block inference.

Measuring ROI: When Few-Shot is Worth It

Few-shot prompting adds cost and complexity. Quantify the tradeoff by measuring accuracy gains against increased token cost and latency overhead.

The chart shows the *relative* cost impact of few-shot across common tasks (illustrative — heights are not measured percentages). Longer example blocks and structured tasks tend to cost more per call.

Decision framework (starting point): weigh the accuracy gain against the added cost per call — then multiply by the *business cost of an error*. For mission-critical tasks, even a small accuracy gain can justify a large cost increase; for high-volume, low-stakes tasks, the bar is much higher.



Key Takeaways: Few-Shot Mastery

Strategic Example Selection

Quality trumps quantity. Curate diverse, representative examples covering edge cases. Use 5-7 examples for optimal balance.

Ordering Matters

Example sequence impacts performance. Experiment with similarity-based ordering for best results, especially on diverse inputs.

Leverage Counter-Examples

Explicitly show what NOT to do. Counter-examples reduce false positives and clarify task boundaries effectively.

Maintain Consistency

Format all examples identically. Inconsistent formatting confuses the model and degrades performance by 5-10%.

Measure Everything

Track accuracy, per-class metrics, variance, latency, and cost. Make data-driven decisions about few-shot investments.

Production-Ready Patterns

Version examples, implement fallbacks, cache prompts, and plan for distribution shifts. Treat examples as critical code.

Thanks